

Face Detection Benchmark: Classical CNN vs Quanvolutional QNN

Stage 1 of a one-shot face recognition system. Faces from LFW, non-faces from CIFAR-10, 32×32 grayscale patches.

Main result (controlled study, 5 seeds, [section 5](#)). Matched for size and training, the quantum layer performs the same as a classical layer of the same shape: balanced accuracy 0.975 (trained QNN) vs 0.974 (matched CNN), and 0.962 vs 0.962 when both are left random. Training adds +0.013 accuracy to the circuit and +0.012 to the classical layer, about the same.

- Random quantum features vs random classical features of the same shape: no clear difference.
- Trained quantum layer vs trained classical layer of the same shape: no clear difference.
- Training the circuit vs leaving it random: helps on both accuracy and 1:10 precision.
- So on this task the 4-qubit layer behaves like a classical layer with the same number of outputs. The large gap in the single run came from the original CNN's much bigger classifier head, not from the quantum layer.

Single run (seed 42, original models): what happens at a realistic class ratio

0.963 → 0.586

QNN precision, balanced →
1:10 background

0.998 → 0.957

CNN precision, balanced →
1:10 background

404 vs 27

False alarms at 1:10 (QNN vs
CNN)

0.57

Best single-statistic AUC (0.5 =
chance). Benchmark passes the
validity gate

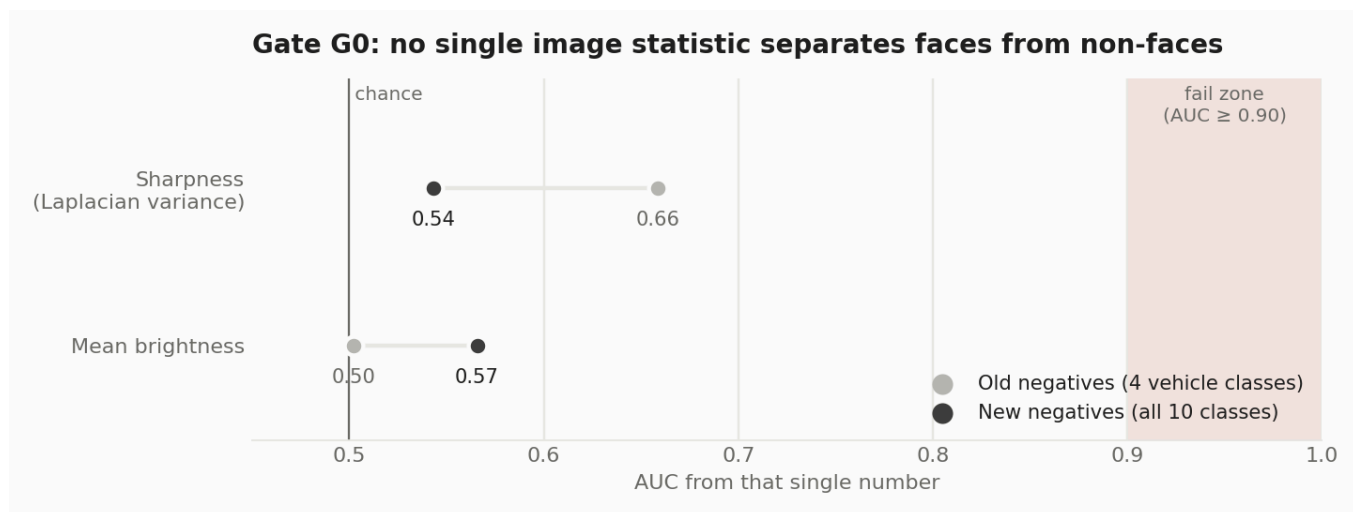
1. Setup

- **Faces:** LFW, every identity with at least 20 photos: 62 people, 3,023 images.

- **Non-faces:** CIFAR-10, all 10 classes, matched 1:1. 80/20 stratified split (4,836 train / 1,210 test), seed 42.
- **CNN:** Conv 3×3 (16 channels) → max-pool → dense 16 → 2 classes. 65,746 trainable parameters.
- **QNN:** 4-qubit quanvolution over 2×2 patches (RY encoding, 2-layer ring of CNOTs, Pauli-Z readout, circuit depth 15) → max-pool → dense 16 → 2 classes. 4,146 trainable parameters plus 24 fixed circuit parameters.

2. Is the benchmark valid? (Gate G0)

Faces and non-faces come from different datasets, so a model could score well by recognising *which dataset* an image came from instead of whether it contains a face. Before any training, three trivial baselines are run. If one number per image (sharpness or brightness) could separate the classes, the accuracy numbers would be meaningless.

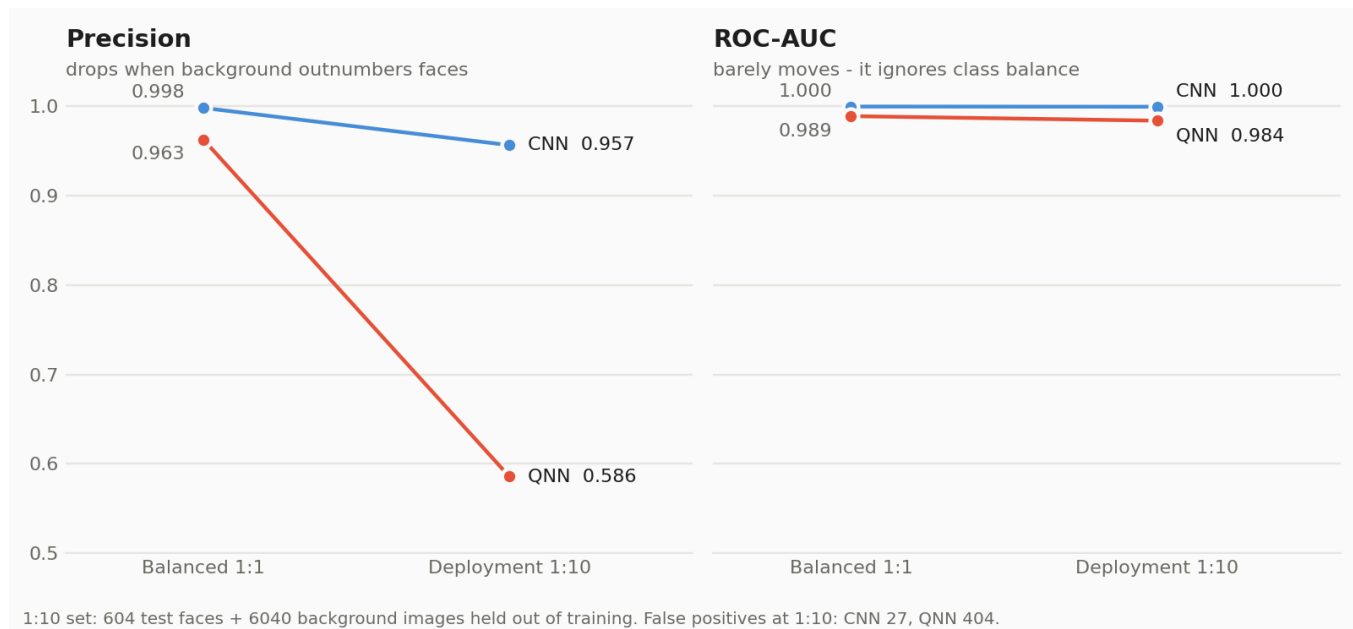


Both statistics stay close to chance. Widening the negatives from 4 vehicle classes to all 10 classes reduced the sharpness signal further (AUC 0.658 → 0.543, Cohen's d 0.63 → 0.29).

A logistic regression on raw pixels does reach AUC 0.992. That is expected for a real task with spatial structure and is not a confound: the gate's self-test (`tests/test_gate_g0.py`) builds two classes with identical sharpness and brightness that differ only in layout, and raw pixels separate those perfectly too.

3. Performance at a realistic class ratio (single run)

A sliding-window detector sees far more background than faces. The models were therefore also tested on **604 test faces + 6,040 background images** (1:10). None of the background images were used in training or in the balanced test set.

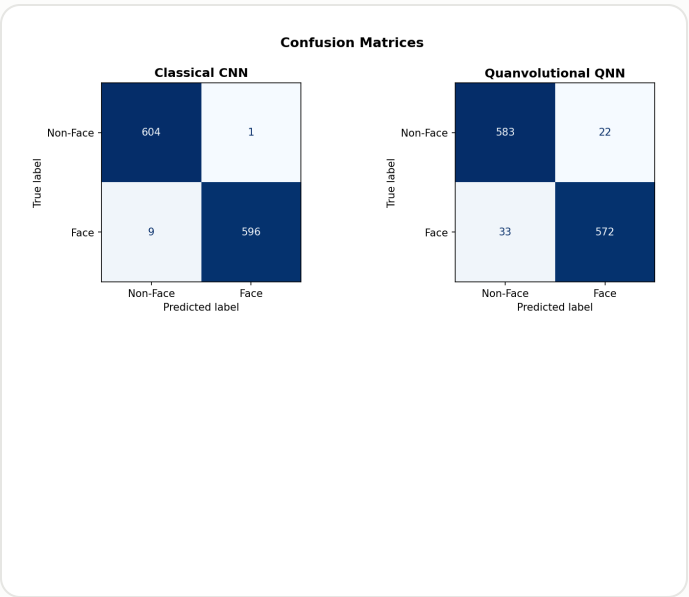
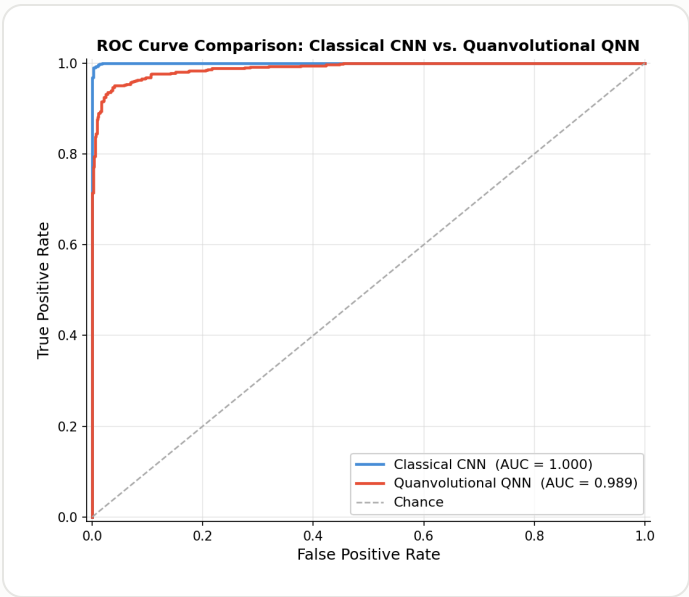


At 1:10	● CNN	● QNN
Precision	0.957	0.586
Recall	0.985	0.945
F1	0.971	0.723
Average precision	0.996	0.920
ROC-AUC	1.000	0.984
False positives (rate)	27 (0.0045)	404 (0.0669)

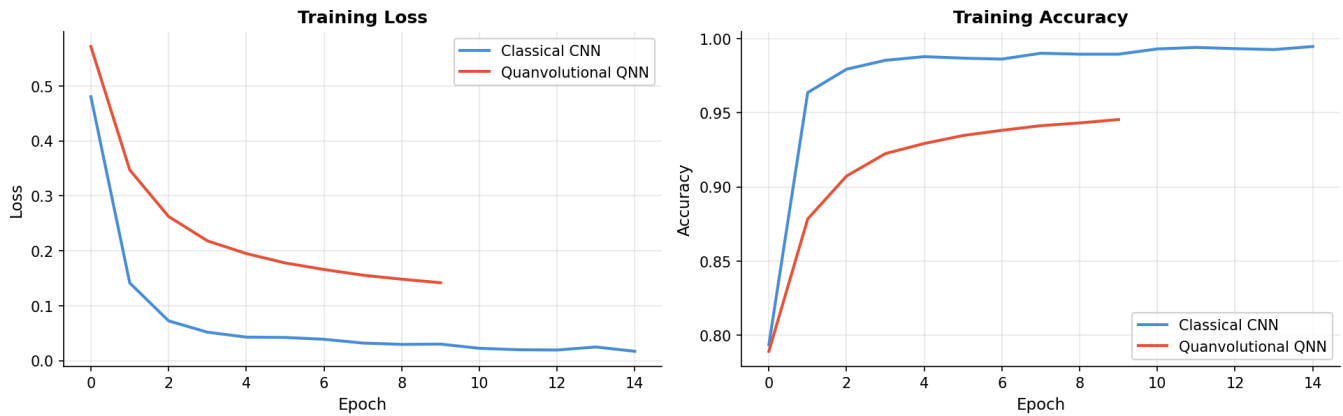
Why ROC-AUC hides this: ROC-AUC depends only on how the model ranks faces against non-faces, not on how many of each there are. Precision does depend on it. The QNN's false-positive rate of 6.7% looks harmless at 1:1 but turns into 404 false alarms when background outnumbered faces ten to one. For a detector, precision and average precision are the numbers to report.

4. Balanced test set (1:1, 1,210 images) (single run)

Metric	CNN	QNN
Accuracy	0.992	0.955
Precision	0.998	0.963
Recall	0.985	0.945
F1	0.992	0.954
ROC-AUC	1.000	0.989
Trainable parameters	65,746	4,146 (+24 fixed)
Inference per 32×32 image (simulator)	0.76 ms	3,940 ms



Training Progress: CNN vs. QNN

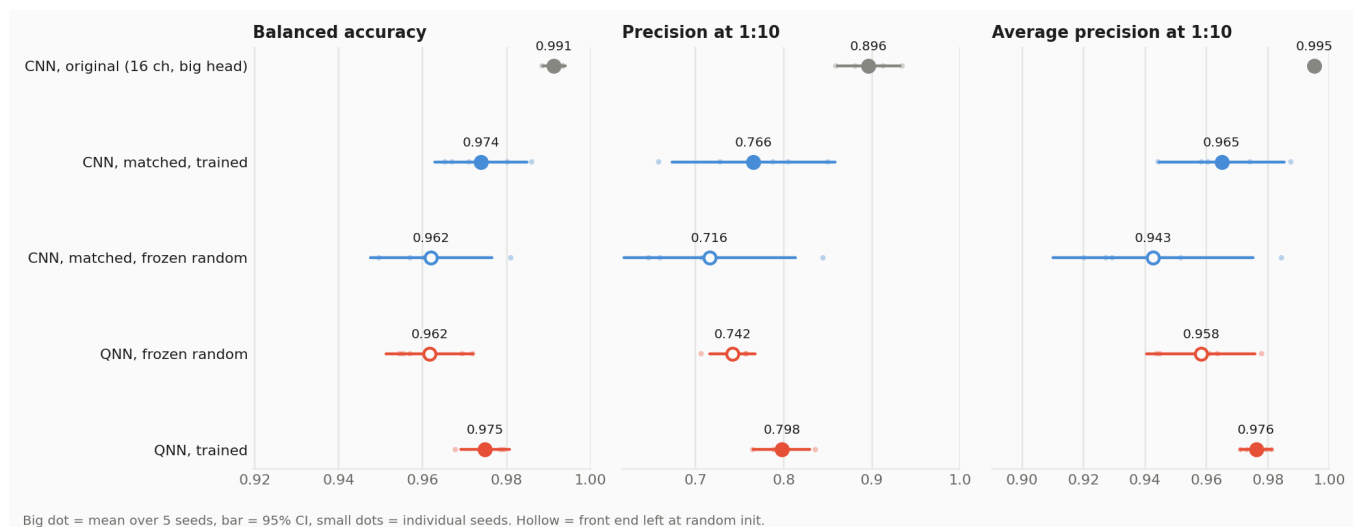


QNN latency is classical simulation time on a CPU, not quantum hardware time.

5. Controlled study: 5 seeds, matched capacity, trained circuit

The single run above cannot attribute the gap to the quantum layer: the models differ in head size and epochs, the circuit is never trained, and there is one seed. This study fixes all three.

- **Matched capacity.** The matched CNN has the QNN's exact shape: a 2×2 , stride-2, 4-channel front end (one output per patch per channel, like the 4 qubits), tanh to keep outputs in $[-1, 1]$ like a Pauli-Z expectation, then the same pool and the same $256 \rightarrow 16 \rightarrow 2$ head. Front ends: 20 classical parameters vs 24 quantum.
- **Trained circuit.** The circuit is re-implemented as a PyTorch statevector so gradients reach it. It matches PennyLane to 5×10^{-7} in value and to 3×10^{-7} in gradient against PennyLane's parameter-shift rule. In the trained arm the circuit parameters moved by up to 0.47 rad, so gradients do reach the circuit.
- **Same training for every model.** Adam, lr 10^{-3} , batch 64, 12 epochs; each model keeps the epoch with the lowest loss on a 10% validation split.
- **Variance.** Each seed is a different train/validation/test split and initialisation. Values are mean $\pm$ 95% confidence interval over seeds.



Model	Trainable params	Balanced acc.	ROC- AUC	1:10 precision	1:10 avg. precision	1:10 false pos.
CNN, original 16 ch 3×3 conv, 4,096- input head (reference)	65,746	0.991 ± 0.003	1.000 ± 0.000	0.896 ± 0.036	0.995 ± 0.001	70 ± 27
CNN, matched 2×2 conv, 4 ch, stride 2, trained	4,166	0.974 ± 0.011	0.996 ± 0.003	0.766 ± 0.092	0.965 ± 0.020	186 ± 97
CNN, matched, frozen same conv left at random init	4,146	0.962 ± 0.014	0.993 ± 0.004	0.716 ± 0.097	0.943 ± 0.033	237 ± 100
QNN, frozen 4-qubit circuit left at random init	4,146	0.962 ± 0.010	0.993 ± 0.004	0.742 ± 0.026	0.958 ± 0.018	201 ± 27
QNN, trained 4-qubit circuit trained end-to-end	4,170	0.975 ± 0.006	0.997 ± 0.000	0.798 ± 0.032	0.976 ± 0.005	149 ± 30

Paired comparisons (same splits; difference = first minus second)

Question	Balanced acc.	1:10 precision	1:10 avg. precision
Random quantum features vs random classical features 5 paired seeds	+0.000 ± 0.022 no clear difference	+0.026 ± 0.106 no clear difference	+0.016 ± 0.044 no clear difference
Trained quantum layer vs trained classical layer (same shape) 5 paired seeds	+0.001 ± 0.009 no clear difference	+0.032 ± 0.064 no clear difference	+0.011 ± 0.021 no clear difference
Does training the circuit help? 5 paired seeds	+0.013 ± 0.009 first is higher	+0.056 ± 0.043 first is higher	+0.018 ± 0.016 first is higher

"No clear difference" means the 95% interval of the paired difference includes zero.

Bottom line.

- Random quantum features vs random classical features of the same shape: no clear difference.
- Trained quantum layer vs trained classical layer of the same shape: no clear difference.
- Training the circuit vs leaving it random: helps on both accuracy and 1:10 precision.
- So on this task the 4-qubit layer behaves like a classical layer with the same number of outputs. The large gap in the single run came from the original CNN's much bigger classifier head, not from the quantum layer.

6. Limitations

- **Sections 2–4 are a single run** (seed 42, original models). Use the controlled study in section 5 for any CNN vs QNN claim.
- **Simulated, noiseless circuit.** All quantum results are exact statevector simulation on a CPU. Hardware noise would only lower the QNN numbers.
- **Small circuit.** 4 qubits, 2 layers, 24 parameters. Nothing here says how larger circuits behave.
- **Balanced training only.** Every model trains at 1:1 and is tested at 1:10. Training at a skewed ratio, where focal loss would become relevant, has not been tried.
- **Patch classification, not full detection.** The task classifies 32×32 patches; there is no detection on whole images yet.

7. Next steps

1. Train at the skewed ratio and compare plain cross-entropy with focal loss.
2. Add depolarising noise to the circuit to see how much of the QNN result survives realistic hardware.
3. Scale the circuit (more layers, larger patches) with the matched classical counterpart scaled alongside.
4. Run as a sliding-window detector on full images (e.g. WIDER FACE) and report average precision.

8. Reproduce

- `python main.py` : full run (G0 gate, both models, balanced and 1:10 evaluation, all figures). Console output of the reported run is in `outputs/run_log.txt`.
- `python -m src.baseline_g0` : validity gate, old vs new negatives.
- `python tests/test_pipeline.py` and `python tests/test_gate_g0.py` : checks that train, test and background sets never overlap, and that the gate catches known confounds.
- `python study.py` then `python -m src.study_report` : the controlled 5-seed study and its summary (`outputs/study_runs.jsonl` , `outputs/study_summary.json`).

- `python make_presentation.py` : rebuilds this page from `outputs/` .